NovaLang: Final Project Report

CS 609 | Part 6 | October 2026

1. Language and purpose

NovaLang is a small interpreted language created to demonstrate how a programming language turns source text into behavior. It is intended for short examples that make core language concepts visible: expressions, control flow, functions, scope, and recursive calls. A NovaLang program follows the pipeline:

```
Source → Lexer → Tokens → Parser → AST → Interpreter → Output
```

The project was designed cumulatively. Part 1 defined the language and grammar; Part 2 implemented tokenization; Part 3 added parsing and an abstract syntax tree; Part 4 made the tree executable; and Part 5 added functions, scope, and recursion. Part 6 integrates these stages and prepares final testing and documentation.

2. Language design decisions

NovaLang uses `let name = expression;` for declarations and `name = expression;` for reassignment. A semicolon terminates a statement; braces delimit a block. `print(expression);` writes a value. Conditions use `if (expression) { ... } else { ... }` and `while (expression) { ... }`. Function definitions use the original `fn name(parameters) { ... return expression; }` form. The lexer accepts `function` as an alias used in later examples.

The language supports integer and decimal numbers, `true` and `false`, arithmetic (`+`, `-`, `*`, `/`, `%`), comparisons (`==`, `!=`, `<`, `>`, `<=`, `>=`), and logical operators (`&&`, `||`, `!`). Comparison results are Booleans, and `if` and `while` conditions must evaluate to a Boolean. Logical conjunction and disjunction short-circuit. The initial Part 1 proposal was Boolean values and logical operators. During Part 5, the group selected and implemented recursion as its extension. Both capabilities remain in the integrated language; recursion is the final project's selected extension feature.

The grammar makes precedence explicit. From highest to lowest, it is parentheses; unary `!` and `-`; multiplication, division, and modulo; addition and subtraction; relational comparisons; equality; `&&`; and `||`. Binary operators at a precedence level associate left to right. For example, `2 + 3 * 4` becomes `+(2, *(3, 4))` and evaluates to `14`.

3. Grammar, lexer, and parser

The canonical grammar is maintained in `grammar.ebnf`. It defines a program as statements and includes declarations, assignments, print statements, blocks, conditionals, loops, function definitions and calls, returns, and expressions. This excerpt shows statement alternatives and expression precedence:

```
<statement> ::= <variable-declaration> | <assignment> | <print-statement>
              | <if-statement> | <while-statement> | <function-definition>
              | <return-statement> | <expression-statement>
<expression> ::= <logical-or>
<logical-or> ::= <logical-and> { "||" <logical-and> }
<logical-and> ::= <equality> { "&&" <equality> }
<term> ::= <factor> { ( "+" | "-" ) <factor> }
<factor> ::= <unary> { ( "*" | "/" | "%" ) <unary> }
```

These layered productions make multiplication bind tighter than addition, while comparisons bind tighter than equality and logical operators. Primaries include literals, variables, calls, and parenthesized expressions.

The lexer scans from left to right. It recognizes identifiers and keywords, integer and decimal literals, one-character punctuation/operators, and two-character comparison/logical operators. It skips whitespace and stores each token's type, lexeme, line, and column. Invalid characters and malformed numeric literals raise a lexical error instead of reaching Python as an unexplained exception. The `--tokens` command-line mode exposes token kinds and locations for inspection. For example, `fn add(a, b) { return a + b; }` begins with `FN`, `ID(add)`, `LPAREN`, `ID(a)`, and `COMMA`; each token also carries its line and column.

The parser is recursive descent. Its methods correspond to grammar layers and consume tokens in order. Nested expression parsers preserve precedence. A syntax error reports the unexpected token and its source position where available. `--ast` prints the parsed tree without execution.

4. AST, interpreter, and scope

The AST uses node classes that represent program structure rather than punctuation. `ProgramNode` contains statements; `NumberNode`, `BooleanNode`, and `VariableNode` represent values and references; `UnaryOpNode` and `BinaryOpNode` represent expressions; and statement nodes represent declarations, printing, blocks, conditionals, loops, functions, calls, and returns. Parsing `let result = 2 + 3 * 4;` produces this shape:

```
AssignmentNode(Declaration)
  VariableNode(result)
  BinaryOpNode(+)
    NumberNode(2)
    BinaryOpNode(*)
      NumberNode(3)
      NumberNode(4)
```

Multiplication is nested below addition, so it is evaluated first. The full printed tree is available with `python3 main.py --ast tests/programs/ast_example_2.nova`.

The interpreter walks the AST and evaluates each node. It stores output values as text and prints them. Arithmetic and comparison operators check operand types; comparisons return Python Boolean values, displayed as `true` or `false`. A division operation that produces a zero divisor is rejected. Conditions reject numeric values rather than treating them as truthy or falsey. Logical operators evaluate the right operand only when necessary.

`Environment` is the variable symbol table. Each environment maps names to values and may refer to a parent. Lookup checks the current map before walking upward; assignment updates the nearest existing definition, while `let` defines locally. Blocks create child environments. Each function call creates a fresh local environment with the function's defining environment as parent, binds parameters, runs the body, and returns through an internal control signal. For example, global `x = 10` stays unchanged when a function call creates local parameter `x = 20`; the function prints `20`, then the caller prints `10`.

5. Extension, errors, and testing

Recursion is demonstrated by a factorial function. It stops at a base case and otherwise calls itself with a smaller argument. For `factorial(5)`, the calls eventually reach `factorial(1)` and return outward, producing `120`. Each call gets its own parameter binding, so one call's `n` does not overwrite another's. The existing examples also demonstrate ordinary functions, parameters, return values, global and local variables, shadowing, and multiple calls.

Errors are handled at the stage that can identify them. The lexer reports invalid characters and malformed numbers. The parser reports invalid syntax with token locations when possible. The runtime reports undefined variables, undefined functions, wrong argument counts, division by zero, non-Boolean conditions, invalid operand types, and returns used outside functions. Lexical and parse errors include line and column; runtime diagnostics currently report the semantic problem without a source location.

Testing is organized around complete NovaLang programs and focused unit tests. Twelve Part 6 valid programs cover each required area and an integrated example; thirteen additional programs provoke distinct errors. Expected output files are under `tests/part6/outputs/`.

Program	Coverage	Program	Coverage
01 arithmetic	Numeric operations	07 while sum	Loop and reassignment
02 precedence	Precedence, parentheses, unary	08 functions	Definitions, calls, returns
03 variables	Declarations and assignment	09 scope	Local/global and shadowing
04 print values	Numbers and Booleans	10 recursion	Factorial extension
05 comparisons	Relational/equality operators	11 logical	Boolean operators, short-circuit
06 if/else	Branching	12 integrated	Combined language features

The error suite includes lexical and syntax errors, undefined variables/functions, division and modulo by zero, invalid operands and conditions, wrong argument counts, return outside a function, duplicate parameters, local-scope failure, and recursion-limit protection. The complete test suite was run with `python3 -m unittest discover -s tests -v` on Python 3.9.6: all 64 tests passed. The syntax check, `python3 -m compileall -q .`, also passed. Actual output for all 12 valid programs and 13 error programs matched the expected files. These results are recorded in `tests/part6/RESULTS.md`. Package markers in nested test directories ensure recursive standard-library discovery includes the full suite.

6. Challenges, lessons, contribution statement, and AI use

An integration challenge was keeping the original Part 1 `fn` spelling consistent with the later Part 5 `function` spelling. The final lexer accepts both. The implementation also had to run under the available Python 3.9.6 baseline; postponing evaluation of a union type annotation avoids a runtime import failure in that version. During integration, recursive `unittest` discovery initially omitted tests nested in folders without package markers. Adding those markers made the final test command reach the full tree. These issues show why cumulative design choices and test discovery need direct integration checks.

Individual contribution statement

The project responsibilities were divided across the six cumulative milestones as follows.

Group member	Project parts	Contribution summary
Sarada Garlapati	Parts 1 and 3	Language design and grammar; parser and AST
Krishna Thakar	Parts 2 and 6	Lexer; final integration, testing, report, and demonstration preparation
Rabin KC	Parts 4 and 5	Interpreter and control structures; functions, scope, and recursion

AI use

The project used OpenAI ChatGPT assistance during earlier design, lexer, parser, interpreter, test, and documentation milestones; the milestone-specific history is retained in `AI_USE_STATEMENT.md`. For Part 6, GPT-6.1 Sol supported implementation and tests, GPT-6 Luna supported documentation, and GPT-6 Astra advised on design and review. One concrete integration correction was postponing evaluation of a union annotation for Python 3.9.6; the integration also fixed nested `unittest` discovery with package markers and restored `fn` while retaining `function` as an alias. AI-assisted changes were verified through the complete automated test suite, expected-output comparisons, Python syntax checks, and an independent technical review.

Repository: github.com/krsnathkr/NovaLang, a private repository. The completed Part 6 implementation is available on the `codex/part6-final` branch.